A

Mini Project Report

on

DataPrepX – AI Enhanced Survey Data Preparation System

Submitted in partial fulfillment of the requirements for the

degree

Third Year Engineering – Computer Science Engineering (Data Science)

by

Harsh Lad	**24207002**
Siddhesh Surve	**24207004**
Aryan Shelar	**24207012**
Sujal Jain	**24207015**

Under the guidance of

Prof. Priyanka Karhade

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on **DataPrepX – AI Enhanced Survey Data Preparation System** has been submitted by Harsh Lad 24207002, **Siddhesh Surve 24207004** , **Aryan Shelar 24207012** and **Sujal Jain 24207015** who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science Engineering (Data Science)**, during the academic year **2024-2025** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Prof. Priyanka Karhade
Guide

Dr. Pravin Adivarekar
HOD, CSE (Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:
1.

Internal Examiner:
1.

Place: A. P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide Prof. Priyanka Karhade. Expressing gratitude towards our HoD, **Dr. Pravin Adivarekar**, and the Department of Computer Science Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Prof. Avani Nair** who gave us his/her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

Abstract

1. Introduction……..6

   1.1.Purpose.....7

   1.2.Problem Statement……...8

   1.3.Objectives..... 10

   1.4.Scope.....…...12

2. Literature Review…….....14

3. Proposed System…….....21

   3.1. Features and Functionality……... 21

4. Requirements Analysis..... 27

5. Project Design..... 31

   5.1.Use Case diagram…… …... 33

   5.2.DFD (Data Flow Diagram) …… 36

   5.3.System Architecture…….31

   5.4.Implementation…….40

6. Technical Specification……..…….....44

7. Project Scheduling.....49

8. Results……..…….52

9. Conclusion…….....57

10. Future Scope…….58

References

ABSTRACT

The digitization of survey data collection has created a systemic challenge: raw datasets invariably contain missing values, outliers, inconsistent encodings, and sampling biases that degrade analytical reliability. Traditional preprocessing is manual, consumes 70–80% of project time, requires programming expertise, and provides no mechanism for quantifying correction reliability.

DataPrepX is a full-stack, AI-enhanced survey data preparation system built on a Python Flask REST API backend and a React/Tailwind CSS frontend. It delivers a three-phase workflow — authenticated upload, configurable preprocessing, and results with downloadable outputs. The core DataProcessor class executes a five-stage pipeline: missing value imputation (Mean, Median, or KNN strategies via scikit-learn), IsolationForest outlier detection with configurable contamination, rule-based domain validation, survey weight application, and weighted statistical estimation with 95% confidence intervals.

The system's key innovation is the AI Confidence Calibration module, which assigns a mathematically derived confidence score (0–100%) to every individual correction — a first-of-its-kind feature. Imputation scores combine distribution fit ($\exp(-z^2/2)$) with method reliability; outlier scores derive from Isolation Forest anomaly scores; rule violation corrections receive a fixed 92%. Six visualizations are generated server-side at 300 DPI and served through JWT-authenticated endpoints. All outputs sync to Google Cloud Storage.

Testing on the Adult Census Income Dataset (48,842 records) demonstrated pipeline completion in 6–9 seconds, 95% data retention, and a 95% confidence interval of ±0.041 hours for the hours-per-week target variable.

Keywords: *Survey Data Preprocessing, Missing Value Imputation, Isolation Forest, AI Confidence Calibration, Flask, React, JWT, KNNImputer, Weighted Mean, Google Cloud Storage*

Chapter 1

Introduction

The global reliance on survey-based data collection has never been more acute. National governments depend on surveys like the Periodic Labour Force Survey (PLFS), the National Sample Survey (NSS), and the Annual Survey of Industries to make trillion-rupee budgetary decisions. Healthcare authorities rely on National Family Health Survey (NFHS) datasets to assess maternal mortality, child malnutrition, and immunization coverage. Academic institutions and independent researchers build entire fields of knowledge on secondary survey data obtained from global repositories. The volume, velocity, and variety of survey data being generated annually has increased by orders of magnitude over the past two decades, driven primarily by the digitization of field data collection using mobile devices, online survey platforms, and automated monitoring systems. Yet, the quality of this data has not kept pace with its quantity. Raw survey data, regardless of how meticulously it is designed and collected, arrives at the analyst's desk in a state of fundamental unreliability. Missing values arising from respondent non-response, interviewer recording errors, and network connectivity failures during digital submission are universal. Outliers introduced by genuine extreme measurements, calibration errors in automated sensors, or deliberate misreporting by survey respondents are endemic. Inconsistent categorical encodings, duplicate records introduced by submission retries, and field values that violate fundamental domain logic — negative ages, income values below the legal minimum wage, implausible geographic coordinates — further degrade dataset integrity. The field of data preprocessing has developed a rich toolkit of algorithms and techniques to address these challenges. However, the deployment of this toolkit in real-world survey analysis contexts is severely hampered by two structural barriers. First, meaningful preprocessing requires programming expertise — the ability to write Python, R, or SAS scripts, to select appropriate algorithms, to tune hyperparameters, and to validate outputs. This expertise is simply absent in most domain expert communities: the epidemiologist, the labor economist, and the rural development officer who work with survey data daily are not software engineers. Second, and critically, existing preprocessing tools provide no mechanism for users to understand the reliability or confidence of the automated corrections they apply. A KNN-imputed value in a column with 2% missingness and strong inter-feature correlations is vastly more reliable than a mean-imputed value in a column with 45% missingness and near-zero correlations with other features — yet every existing tool treats both corrections identically, presenting them without any uncertainty quantification. DataPrepX is engineered to systematically dismantle both barriers. It delivers a fully automated, configurable preprocessing pipeline through a web interface that requires zero programming knowledge. And it introduces a novel AI Confidence Calibration system that assigns mathematically derived confidence scores to every individual correction, creating an unprecedented level of transparency and user control in automated data preprocessing

1.1 Purpose:

The primary purpose of DataPrepX is to democratize access to high-quality survey data preprocessing by eliminating the programming and statistical expertise barriers that have historically made this process the exclusive domain of trained data scientists. The system is designed as a production-grade, full-stack web application that transforms the traditionally manual, multi-day activity of survey data preparation into a guided, automated, minutes-long workflow.

This document serves as the comprehensive technical and functional specification for the DataPrepX system, defining its architecture, algorithms, interface design, and evaluation methodology. Its target audience encompasses three primary user groups.

The first group is government data analysts and statistical officers at institutions such as MoSPI, state statistical bureaus, and public health departments who must preprocess large, complex survey datasets as part of their official duties but lack the programming background to do so programmatically. For this group, DataPrepX provides a reliable, reproducible preprocessing pipeline with detailed processing logs that serve as an audit trail for official data quality procedures.

The second group is academic and independent researchers who work with publicly available survey datasets from repositories such as the IHDS (India Human Development Survey), the National Family Health Survey (NFHS), and the Demographic and Health Surveys (DHS) program. For this group, DataPrepX eliminates the preprocessing overhead that currently consumes the majority of project time, allowing researchers to focus on the analytical questions that motivated their work.

The third group is data science educators and students who seek to understand the practical implementation of preprocessing algorithms in a real-world context. The system's AI Confidence Calibration module, which exposes the internal confidence calculations of each correction through an API, provides an educational tool for understanding the mathematical basis of imputation reliability and outlier detection confidence.

1.2 Problem Statement:

The core problem that DataPrepX addresses is the fundamental inadequacy of current survey data preprocessing practices in the face of the scale, complexity, and transparency demands of modern data-driven governance and research. This inadequacy manifests across four distinct and inter-related dimensions.

1. The Expertise Barrier and the Democratization Gap: The most effective survey data preprocessing workflows require not just programming proficiency but a nuanced understanding of statistical theory — the ability to distinguish between Missing Completely At Random (MCAR), Missing At Random (MAR), and Missing Not At Random (MNAR) patterns, to make informed choices between parametric and non-parametric outlier detection, and to apply survey design weights correctly in the context of stratified sampling. This level of expertise represents years of training. The vast majority of the 8,000+ statistical personnel employed by the Indian government's statistical system, and the hundreds of thousands of researchers globally who work with survey data, do not possess this expertise. They are forced to either use dangerously simplified approaches — deleting all rows with any missing value, applying a blanket mean imputation without consideration of distribution — or to spend significant budget and time engaging specialized data cleaning consultants. This expertise gap represents a systematic failure mode in the global survey data ecosystem.

2. The Time Cost of Manual Preprocessing: Industry surveys of data scientists consistently find that 60–80% of project time is consumed by data preparation activities. For survey specific research, this proportion is even higher due to the additional complexity of survey specific issues: cleaning inconsistent response codings, validating skip-pattern logic, merging multi-module survey instruments, and applying post-stratification weights. A research team working with a 50,000-record, 200-variable survey dataset may spend 3–4 weeks on preprocessing before a single analytical model can be run. DataPrepX's automated pipeline, which processes the same dataset in under 15 seconds, represents not a marginal improvement but a qualitative transformation of the research workflow — the difference between a week of cleaning and an afternoon of configuration.

3. The Opacity of Automated Corrections: Existing automated preprocessing tools — from pandas' fillna() and scikit-learn's Simple Imputer to commercial tools like Alteryx and

KNIME — share a fundamental design flaw: they provide no uncertainty quantification for their corrections. When KNN imputation fills in a missing income value with INR 47,350, the tool does not tell the analyst whether this value was imputed with 91% confidence based on five highly similar neighbors with low income variance, or with 43% confidence based on five dissimilar neighbors spanning a range of INR 15,000 to INR 2,00,000. Both corrections look identical in the output CSV. This opacity is not merely inconvenient — it is analytically dangerous. Downstream models trained on low-confidence imputations may produce spurious patterns that appear statistically significant but are artifacts of the imputation process itself. DataPrepX's AI Confidence Calibration module addresses this fundamental gap by making the reliability of every correction visible, quantified, and actionable. 4. The Fragmentation of the Preprocessing Workflow: Today's data preprocessing landscape is defined by fragmentation. Analysts use one tool for missing value imputation (e.g., scikit-learn's imputers), a different tool for outlier detection (e.g., PyOD or custom IQR scripts), yet another tool for statistical estimation (e.g., the R survey package or Python's samplics), and a completely separate environment for visualization (e.g., matplotlib, seaborn, or Tableau). Each transition between tools requires file export, format conversion, and manual integration of results — introducing opportunities for errors, losing provenance information, and further fragmenting the audit trail. DataPrepX unifies the entire workflow — from file upload through imputation, outlier detection, rule validation, weighted estimation, visualization generation, and downloadable report — within a single, integrated platform accessible through a web browser

1.3 Objectives:

To systematically address the sophisticated challenges defined in the problem statement, this project is guided by a rigorous, foundational set of actionable objectives. These eight core objectives govern the entire system architecture, from the design of individual algorithm classes to the structure of the API and the user interface workflow.

Objective 1: Configurable Multi-Strategy Missing Value Imputation

The system must implement three distinct imputation strategies selectable by the user at runtime. Mean Imputation applies scikit-learn's SimpleImputer with strategy='mean' to fill missing numeric values with the column arithmetic mean — computationally efficient and suitable for normally distributed features with low missingness rates (below 10%). Median Imputation applies SimpleImputer with strategy='median', replacing missing values with the column median — robust to skewed distributions and the presence of outliers that would inflate the mean, making it the preferred choice for income, expenditure, and other rightskewed survey variables. KNN Imputation applies scikit-learn's KNNImputer with n_neighbors=5 to impute missing values based on the weighted average of the five most similar records — capturing inter-feature correlations that purely univariate methods miss. Categorical columns in all three modes receive mode imputation. The system must also automatically detect and drop completely empty columns before imputation to prevent degenerate cases.

Objective 2: Isolation Forest-Based Outlier Detection

The system must integrate scikit-learn's IsolationForest algorithm with a configurable contamination parameter (defaulting to 0.05 — 5% of records flagged as outliers) applied simultaneously across all numeric columns. Unlike univariate methods such as IQR filtering or Z-score thresholding that evaluate each feature independently, Isolation Forest identifies records that are anomalous in the joint feature space — capturing multi-dimensional outlier patterns that simpler methods miss. Detected outliers must be replaced with column medians rather than deleted, preserving sample size while controlling the influence of extreme values. The algorithm's raw anomaly score must be captured for each outlier record and column for use in the confidence calibration system.

Objective 3: Domain-Specific Rule-Based Validation

The system must support configurable range validation rules for specified columns. Rules enforce minimum and maximum bounds on field values — for example, constraining age fields to positive values, income fields to non-negative values, or test scores to valid ranges. Violations must be corrected by clipping to the nearest valid bound, with the original and corrected values logged for the audit trail. The validation rule system must be extensible without requiring code changes, accepting rule definitions as configuration parameters in the API request payload.

Objective 4: AI Confidence Calibration Module (Core Innovation)

The system must implement a novel multi-factor confidence scoring mechanism assigning a score between 0% and 100% to every individual data correction. For imputation corrections, the confidence formula must incorporate both the statistical fit of the imputed value within the column's distribution (via a Gaussian-based distribution fit metric) and the inherent reliability of the imputation method. For outlier corrections, the confidence must be derived from the Isolation Forest's anomaly scoring function. The system must provide a dedicated API endpoint for retrieving all corrections with their scores, and a separate endpoint for user adjustment of correction acceptance status (auto_accept, manually_approved, manually_rejected). A calibration accuracy metric must be computed using pairwise ranking of approved versus rejected confidence scores.

1.4 Scope

The scope of DataPrepX defines the precise boundaries of its current analytical capabilities, the file formats and data types it is engineered to handle, and explicitly acknowledges its current limitations and areas identified for future development. Defining the scope with precision is critical for setting accurate expectations for operational deployment and academic evaluation.

In-Scope: Supported File Formats.

The system accepts and processes two universally used survey data formats. CSV files (.csv) — the standard exchange format for virtually all survey data repositories, statistical agencies, and data portals. The system uses pandas.read_csv() for parsing, supporting all standard delimiter configurations and encoding schemes. Excel files (.xlsx) — the primary working format for government statistical offices and research teams that maintain and annotate datasets in Microsoft Excel. The system uses pandas.read_excel() via the openpyxl engine. For both formats, the system performs automatic data type inference, converting numeric strings to float64, preserving categorical variables as object dtype, and identifying datetime columns for exclusion from numeric preprocessing steps.

In-Scope: Dataset Characteristics.

The system is designed and validated for tabular survey datasets up to 500MB in file size, with an arbitrary number of rows (tested up to 100,000 records) and up to several hundred columns. Both numeric and categorical variables are fully supported. Mixed-type columns containing numeric values stored as strings are handled through pandas' dtype inference. The system explicitly manages edge cases including completely empty columns (automatically dropped), uniform columns with zero variance (skipped during IQR-based operations to prevent empty DataFrames), and columns with entirely null values after the data type is parsed.

In-Scope: Preprocessing Operations.

Missing value imputation (three strategies for numeric, mode for categorical), Isolation Forest outlier detection and correction, configurable range validation rules, survey design weight application, weighted statistical estimation for both numeric and categorical target variables, and six-type visualization generation. All operations are fully logged with before-and-after statistics for audit trail purposes.

Out-of-Scope.

The system does not currently support binary survey formats (SPSS .sav, Stata .dta, SAS .sas7bdat) which require specialized parsing libraries. Real-time streaming data from CATI systems or IoT sensors is not supported — the system processes static, uploaded files

only. Persistent cross-session user dashboards and historical job retrieval require a database backend (currently in-memory) and are identified for future development. Batch processing of multiple datasets simultaneously, relational database connectivity as a data source, and multi-tenant organizational account management are out of current scope.

Chapter 2

Literature Review

The challenge of automated data quality improvement has attracted significant research attention across statistics, machine learning, and database systems communities. This chapter presents a structured review of the most relevant prior work, organized by the primary technical challenge each contribution addresses, and concludes with a synthesis of the research gaps that motivate the specific design choices made in DataPrepX.

2.1 Missing Value Imputation Approaches

V.S.V.S. Murthy and J. N.V.R. Swarup Kumar [1] presented a comprehensive systematic review in 2025 comparing traditional, machine learning, and deep learning approaches to missing data imputation across the full spectrum of available techniques. The traditional category included Mode Imputation, KNN with One-Hot Encoding for categorical features, Hot Deck imputation, MICE (Multiple Imputation by Chained Equations), and EM (Expectation-Maximization) algorithm-based imputation. The machine learning category encompassed MissForest (a random forest-based imputation method), Bayesian Networks, CatBoost, and LightGBM gradient boosting approaches. The deep learning category included GAIN (Generative Adversarial Imputation Networks), DAE (Denoising Autoencoders), VAE (Variational Autoencoders), RNN-based sequential imputation, and Transformer/BERT-like contextual imputation architectures. All methods were evaluated on the Wild Bird HPAI dataset using F1-score, precision, and recall.

The study's central finding was that while deep learning methods — particularly GAIN and Transformer-based approaches — achieved the highest imputation accuracy metrics, they require substantial computational resources (GPU clusters for training times measured in hours), present significant challenges in scalability to high-dimensional survey datasets, and offer limited interpretability for domain expert validation. Traditional and simpler ML methods, though less accurate, remain the practical choice for real-world deployment. DataPrepX directly addresses this trade-off by providing both computationally efficient Mean/Median imputation for quick deployment and KNN imputation for superior accuracy

when the dataset structure supports it, without requiring the user to understand the underlying mathematical trade-offs.

Alaa Tharwat and Wolfram Schenck [2] proposed an active learning framework for handling missing data in 2025, introducing a novel multiple imputation method that selects optimal feature groups based on importance scores to quantify imputation uncertainty. The method combined a two-phase active learner (exploration phase for representativeness and exploitation phase for informative boundary sampling) with explicit accounting for imputation uncertainty to avoid querying imprecisely imputed data points. The system was tested on binary and multiclass datasets with varying missing rates from 10% to 50%. The primary limitations of this approach are its restriction to real-valued (numeric) features only — categorical and ordinal features, which are ubiquitous in survey datasets, are not handled — and its computational intensity, which makes it impractical for large survey datasets without dedicated computational infrastructure.

Muhammad Ishaq et al. [3] developed ensemble models using the ECOC (Error Correction Output Codes) framework with SVM, KNN, and a hybrid SVM+KNN+MLP classifier specifically for imputing missing values in categorical datasets in 2024. The approach was tested on CPU performance, Hypothyroid, and Breast Cancer datasets under different missing patterns including MCAR, MAR, and MNAR. While the ensemble approach demonstrated improved accuracy over single-model imputers, it requires substantial labeled training data for the imputation models themselves — a requirement that defeats the purpose in scenarios where the dataset under analysis is the only available data. Performance varied significantly across datasets and missing data patterns, and the computational complexity of the ensemble significantly exceeded simpler imputation methods.

2.2 Outlier Detection Techniques

The Isolation Forest algorithm, first introduced by Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou [6] in 2008, represents the theoretical foundation of DataPrepX's outlier detection module. The Isolation Forest operates on a fundamentally different principle from distancebased or density-based outlier detection methods: rather than profiling normal observations and identifying deviations, it explicitly isolates anomalies through random recursive partitioning. In a dataset where normal observations are clustered and numerous, they require many random splits to be isolated from the rest of the data. Anomalous observations, being few and statistically different from the bulk, can be isolated in very few random splits.

The algorithm's key advantages for survey data preprocessing are threefold. First, it operates in the joint feature space of all numeric variables simultaneously, identifying records that are anomalous across multiple dimensions — a capability that univariate methods like IQR filtering fundamentally lack. A record with a moderate income and a moderate age individually, but an extreme income-to-age ratio, would be missed by column-wise IQR filtering but correctly identified by Isolation Forest. Second, the contamination parameter provides a direct, interpretable control over the aggressiveness of outlier removal — the analyst can specify what fraction of the dataset they expect to be anomalous based on domain knowledge. Third, the algorithm's anomaly scores provide a continuous measure of outlierness that DataPrepX directly leverages in its confidence calibration system.

Rocco Cassandro, Quing Li, and Zhaojun Steven Li [4] proposed an Entropy-based Data Partition (EDBP) approach in 2023, using Shannon's entropy to identify high-purity data subsets where attribute values link directly to class labels. The approach combined a threepatterns feature visualization technique with Equal Width binning for discretization. While the entropy-based perspective offers theoretical insights, the method requires user-defined thresholds (entropy threshold E_t and volume threshold k_t) that are difficult to set without domain expertise, and the discretization step introduces information loss for

continuous features that is particularly problematic for financial and demographic survey variables.

2.3 Automated Preprocessing Pipelines

Mehwish Bilal and colleagues [5] presented Auto-Prep in 2022, a Python-based automated preprocessing pipeline that automatically detects duplicates, infers data types, identifies missing values and categorical features, and performs imputation, encoding, feature reduction, and scaling. The system recommended optimal preprocessing techniques after evaluating a set of candidate methods on each column and generated a visualization report. Auto-Prep was tested on 10 diverse datasets and demonstrated that full automation of the preprocessing configuration selection is achievable. However, the system has critical limitations that DataPrepX specifically addresses. Auto-Prep still requires user confirmation for complex preprocessing decisions, lacks a web interface accessible to non-programmers, provides no confidence scoring for its corrections, does not support survey-weighted statistical estimation, and was not designed for the specific data quality patterns and weighting requirements of official government survey datasets.

The broader landscape of commercial preprocessing tools — including Alteryx, KNIME, Talend, and Trifacta — offers sophisticated preprocessing capabilities but with two structural limitations that exclude them from the government and academic survey research context. First, they are proprietary and expensive, with licensing costs that are prohibitive for small research teams and government agencies in developing countries. Second, and more critically, none of them provide any mechanism for quantifying the confidence or reliability of automated corrections — the opacity problem identified in the problem statement remains fully unaddressed across the entire commercial landscape.

2.4 Research Gap and Motivation

The systematic review of existing literature and commercial tools reveals a consistent and striking pattern: despite decades of research on individual components of the preprocessing pipeline, no existing open-source or commercial system addresses the complete set of requirements that motivate DataPrepX's design. The specific gaps are:

1. No existing automated preprocessing tool, open-source or commercial, assigns percorrection confidence scores to enable user-driven, informed validation of automated data cleaning decisions. Every reviewed system treats all corrections as equally reliable, regardless of the statistical evidence supporting each individual correction.

2. No unified platform combines the complete preprocessing workflow — imputation, outlier detection, rule validation, survey weighting, statistical estimation, visualization, and reporting — in a web-accessible interface that requires zero programming knowledge.

3. No reviewed system is specifically designed for the unique requirements of official survey data: proper survey design weight application, population-representative statistical estimation, and the audit trail requirements of official statistical reporting.

4. The combination of Isolation Forest (for multi-dimensional outlier detection capturing joint anomalies) with KNN imputation (for relationship-aware missing value filling) within a single, configurable pipeline has not been previously implemented in an accessible web platform

Sr. No	Title	Authors	Year	Methodology	Drawbacks
1.	Missing Data Imputation: A Systematic Comparison of Traditional, Machine Learning and Deep Learning Approaches	V.S.V.S. Murthy, J. N.V.R. Swarup Kumar	2025	Systematic review comparing traditional (Mode, KNN, MICE, EM), ML-based (MissForest, CatBoost, LightGBM), and deep learning-based (GAIN, DAE, VAE, RNN) imputation methods evaluated on the Wild Bird HPAI dataset.	Deep learning methods require substantial computation; challenges in scalability, interpretability. Traditional methods fail to capture complex dependencies.
2.	Active Learning for Handling Missing Data	Active Learning for Handling Missing Data	2025	Novel multiple imputation with featureimportance-based group selection combined with a two-phase active learner. Tested on binary and multiclass datasets with varying missing rates.	Limited to numeric features; computationally intensive multiple imputation; requires careful tuning of exploration–exploitation parameters.
3.	Machine Learning Based Missing Data Imputation in Categorical Datasets	Muhammad Ishaq et al.	2024	Ensemble models using ECOC framework with SVM, KNN, and hybrid (SVM+KNN+MLP) classifier for categorical missing value imputation.	Requires large labeled datasets; risk of overfitting; performance varies significantly by dataset and missing pattern.
4.	An Entropy-based Data Reduction Method for Data Preprocessing	R. Cassandro, Q. Li, Z. S. Li	2023	Three-patterns feature technique with Equal Width binning and Entropy-based Data Partition (EDBP) algorithm using Shannon's entropy to identify high-purity subsets.	Requires discretization and user-defined thresholds; datasetdependent performance; focuses on purity-based reduction only.

5.	Auto-Prep: Efficient and Automated Data Preprocessing Pipeline	Mehwish Bilal et al.	2022	Python-based pipeline that automatically detects and handles duplicates, data types, missing values, encoding, feature reduction, and scaling. Evaluated on 10 diverse datasets	Still requires user input for complex cases; automation may not always select best method for domain-specific datasets; slow on very large datasets.

Table 2.1. Literature Survey of Research Papers

Chapter 3

Proposed System

The proposed system is a production-grade, full-stack web application designed to authenticate digital survey datasets through a sophisticated orchestration of machine learning algorithms, statistical estimation methodologies, and an innovative confidence quantification architecture. The overarching engineering philosophy driving DataPrepX is that automated preprocessing must be transparent — every correction applied to a user's data must carry a mathematically derived measure of its own reliability, transforming a traditionally opaque black box into an auditable, user-controllable pipeline.

The system operates as a unified entity comprising four primary operational layers: the Authentication and Session Management layer that controls access and maintains job isolation; the Data Ingestion and Schema Extraction layer that receives uploaded files and extracts metadata; the AI Processing Core that executes the multi-stage cleaning, calibration, estimation, and visualization pipeline; and the Results Delivery layer that serves processed outputs through authenticated API endpoints and synchronizes files to cloud storage.

The backend is built on Flask, a lightweight WSGI web framework that, combined with Gunicorn's multi-worker configuration, provides the concurrency required for a multi-user deployment. The frontend is a React single-page application that communicates with the backend exclusively through Axios HTTP requests, managing all authentication state clientside through JWT token storage. This architectural separation ensures that the frontend and backend can be independently scaled, deployed, and updated — a critical property for production cloud deployments.

3.1 Features and Functionality

3.1.1 User Authentication and Job Isolation

The authentication layer implements a complete stateless session management system using JSON Web Tokens. When a user registers via POST /api/auth/register, their password is immediately hashed using Werkzeug's generate_password_hash function, which applies the PBKDF2 key derivation function with SHA-256 as the underlying hash and a random salt, producing a cryptographically secure stored credential that cannot be reversed even if the server's memory is compromised. A UUID v4 is generated as the user's persistent identifier. Upon successful login via POST /api/auth/login, Flask-JWT-Extended's create_access_token function generates a signed JWT with a 30-day expiration, encoding the user's email as the identity claim.

All data-related endpoints apply the @jwt_required() decorator, which extracts the JWT from the Authorization: Bearer header and validates its signature and expiration before allowing the request to proceed. Failed validation returns a 401 Unauthorized response. This design ensures complete isolation between users' data: a user's job_id dictionary entries in the processing_jobs in-memory store are only accessible through authenticated requests, preventing cross-user data leakage. Every uploaded file is prefixed with the job's UUID to prevent filename collisions on the server filesystem.

3.1.2 Intelligent Missing Value Imputation

The missing value imputation system, implemented within the DataProcessor.handle_missing_values() method, begins with a pre-processing step that identifies and removes completely empty columns using pandas' dropna(axis=1, how='all') — preventing the imputation algorithms from operating on degenerate features that carry no information. The remaining columns are then categorized by data type using pandas' select_dtypes() method: numeric columns (float64, int64) receive quantitative imputation, while object-type categorical columns receive mode imputation regardless of the configured strategy.

For Mean Imputation, scikit-learn's SimpleImputer(strategy='mean') computes the column arithmetic mean on the non-null values and fills all null positions with this single value. This approach is computationally O(n) per column and statistically appropriate for

normally distributed features with low missingness rates (typically below 10–15%). For Median Imputation, SimpleImputer(strategy='median') uses the column median, which is the 50th percentile of the non-null distribution. The median is immune to the influence of outliers by definition — a single extreme value in a column cannot shift the median as dramatically as it shifts the mean — making it the preferred choice for income, expenditure, and other rightskewed survey variables. For KNN Imputation, scikit-learn's KNNImputer(n_neighbors=5) fills missing values using the weighted average of the five nearest non-missing records, where distance is computed in the joint space of all numeric features. A try-except block wraps the KNN computation to fall back to mean imputation if the imputer fails due to insufficient data.

3.1.3 Isolation Forest Outlier Detection

The outlier detection module, implemented in DataProcessor.detect_outliers(), deploys scikit-learn's IsolationForest with configurable contamination parameter on the full matrix of numeric columns extracted via select_dtypes(include=[np.number]). Unlike column-wise methods that evaluate each feature independently, Isolation Forest fits the ensemble of isolation trees on all numeric columns simultaneously, making it sensitive to multidimensional anomalies — records that are unusual in the joint feature space even when individual feature values appear normal.

The algorithm's fit_predict() method returns binary labels (+1 for inliers, -1 for outliers), and the score_samples() method returns the raw anomaly score — a negative value where more negative scores indicate greater anomalousness. For each record labeled as an outlier (-1), the system iterates over all numeric columns and replaces the record's value in each column with the column median of the original (pre-outlier-correction) dataset, preserving the sample's row while neutralizing its extreme values. The absolute anomaly score for each correction is stored in the correction record and used as the primary input to the AI Confidence Calibration system.

3.1.4 AI Confidence Calibration Module – Core Innovation

The ConfidenceCalibration class represents DataPrepX's most technically novel contribution — a first-of-its-kind uncertainty quantification system for automated data corrections. The architectural motivation is straightforward: an automated system that

makes hundreds or thousands of corrections to a user's data without any indication of how reliable those corrections are is, from a scientific perspective, no better than random manipulation. A researcher who publishes analytical results derived from an opaque, confidence-free preprocessing pipeline cannot defend those results if challenged — they cannot answer the question: 'How do you know those imputations were accurate?'

The confidence scoring system resolves this challenge by assigning every correction a score derived from the mathematical properties of the correction itself and the statistical context of the data. For imputation corrections, the score is computed as: Confidence = (0.4 × distribution_fit + 0.6 × method_score) × 100, where distribution_fit = $\exp(-z^2/2)$ for z = |imputed_value − column_mean| / column_std — a Gaussian-based metric that returns 1.0 for an imputed value exactly equal to the mean and decreases as the value deviates, reflecting the statistical plausibility of the imputed value given the observed distribution. The method_score component encodes the intrinsic reliability of the algorithm: KNN = 0.95 (highest, because it leverages inter-feature relationships), Mean = 0.85, Median = 0.80. For outlier corrections, the confidence score is computed as: Confidence = (0.7 + outlier_score × 0.3) × 100, where outlier_score is the absolute normalized Isolation Forest anomaly score — records with very high anomaly scores (clearly extreme outliers) receive higher confidence corrections than borderline cases. For rule violation corrections, a fixed confidence of 92% is assigned, reflecting the deterministic, domain-validated nature of rule-based decisions.

The record_correction() method logs each correction with a UUID, the correction metadata, the computed confidence score, and a user_action field initialized to 'auto_accept'. The /api/calibration//adjust endpoint allows users to change any correction's action to 'manually_approved' or 'manually_rejected', enabling post-hoc human validation. The _calculate_calibration_accuracy() method computes a pairwise ranking score: for each pair of (approved_correction, rejected_correction), a 'win' is scored if the approved correction has a higher confidence score than the rejected one. The final accuracy = (wins / total_pairs) × 100%, producing a value that measures how well the confidence scores predict user validation decisions — a calibration accuracy of 100% means the confidence scoring perfectly separates corrections users trust from those they reject.

3.1.5 Survey-Weighted Statistical Estimation

The compute_estimates() method handles both numeric and categorical target variables with survey-appropriate weighting methodology. For numeric targets, the weighted mean is computed using numpy.average(values, weights=weights), which implements the formula: $\bar{x}_w = \Sigma(w_i \times x_i) / \Sigma w_i$. The weighted standard error is computed from the weighted variance formula: $Var_w = \Sigma w_i(x_i - \bar{x}_w)^2 / (\Sigma w_i)^2$, followed by $SE = sqrt(Var_w) \times sqrt(n/(n-1))$ to apply the finite sample correction factor. The 95% margin of error is computed as $1.96 \times SE$, and the confidence interval is $[\bar{x}_w - ME, \bar{x}_w + ME]$. The effective sample size — the sum of all weights — is also reported, providing an indication of the dataset's representativeness relative to the target population. For categorical targets, the system uses pandas' groupby on the target variable with the weight column to compute weighted class proportions, identifying the dominant category and reporting the full weighted distribution.

3.1.6 Six-Type Data Visualiation Suite

The VisualizationGenerator class applies a consistent dark theme across all six visualization types: background color #1a1f3a (deep navy), panel color #2d3a5f (lighter navy), accent color #1dd1a1 (teal green), and secondary color #26d0ce (cyan). All figures are rendered using matplotlib's Agg non-interactive backend — mandatory for server-side rendering in headless environments — and saved as 300 DPI PNG files.

The missing_values_heatmap() method uses seaborn's heatmap function on a binary presence/absence matrix of up to 20 columns with the highest missingness rates, using the RdYlGn_r colormap (red = missing, green = present). It computes and displays the overall missing rate as a percentage in the chart title. The confidence_distribution() method generates a histogram of all confidence scores using 20 bins with a vertical mean line overlay, providing the analyst with an immediate visual understanding of whether automated corrections are predominantly high-confidence or span a wide reliability range. The calibration_accuracy_gauge() renders a semicircular gauge with three colored bands (red: 0– 33%, amber: 33–66%, teal: 66–100%) and a needle pointing to the current calibration accuracy percentage. The outlier_detection_summary() creates a vertical bar chart showing the count of outlier corrections per numeric column, ordered by frequency, with the average Isolation Forest anomaly score annotated above each bar. The

correction_types_pie() generates a pie chart with percentage labels showing the proportional split between imputation corrections, outlier corrections, and rule violation corrections. The data_quality_metrics() method produces a grouped bar chart comparing four quality dimensions — Completeness, Validity, Consistency, and Accuracy — before and after preprocessing, with the DataPrepX accent colors distinguishing the two groups.

Chapter 4

Requirements Analysis

4.1 Hardware Requirements

The system's architecture cleanly separates the computational burden between the client (which performs only rendering and UI interactions) and the backend server (which shoulders the entire data processing, algorithm execution, and visualization generation load).

Server Backend (Minimum Requirements for Production Deployment):

- Processor (CPU)
- Memory (RAM)
- Storage
- Cloud Deployment (Google Cloud Run)

Client Frontend (Minimum Requirements):

- Chrome 90+, Firefox 85+, Edge 90+, or Safari 14+. The Tailwind CSS backdrop-filter property used for glass-effect cards requires a browser supporting the CSS backdrop-filter specification. Older browsers render the cards without the blur effect but maintain full functionality.
- A stable internet connection with sufficient upload bandwidth for dataset files. Uploading a 4MB CSV file on a 1Mbps upload connection takes approximately 32 seconds; on a 10Mbps connection, approximately 3 seconds. The preprocessing API call itself is typically under 100KB in both request and response payload size, making network bandwidth the primary constraint only for the initial file upload.

4.2. Software Requirements

The technology stack is purposefully selected to represent current best practices in Python data science operations, modern web development, and cloud-native application deployment.

Backend Execution Environment:

- The backend is OS-agnostic in development (Linux, macOS, Windows 10/11). For production deployment on Google Cloud Run, the container uses a Debian Bookworm-based Python 3.11 slim image, providing the minimal runtime environment necessary for the application while minimizing container image size and attack surface.
- Python 3.10 or higher, required for the syntax features and library compatibility specifications of Flask 3.0, Flask-JWT-Extended 4.6, and scikitlearn 1.5.
- Gunicorn 22.0.0 configured with --workers 2 --threads 4 --timeout 300 for production deployment. The 300-second timeout accommodates large dataset processing pipelines without premature connection termination.
- pandas 2.2.2 for DataFrame operations and file I/O; NumPy 1.26.4 for array computing and weighted statistics; scikit-learn 1.5.0 for SimpleImputer, KNNImputer, and IsolationForest; scipy 1.x for statistical functions; matplotlib 3.8.4 and seaborn 0.13.2 for server-side visualization generation.
- Flask-JWT-Extended 4.6.0+ for JWT token generation and validation; Werkzeug 3.0.3 for PBKDF2+SHA256 password hashing; python-dotenv 1.0.1 for environment variable management from .env files in development.
- google-cloud-storage 2.16.0 for GCS bucket operations; google-auth 2.29.0 for service account credential authentication.

Frontend Client Environment:

- Node.js 18.x or higher and npm for dependency management and build orchestration.
- React 18.2.0 using Create React App 5.0.1 as the build toolchain. React's virtual DOM reconciliation algorithm efficiently handles the dynamic rendering of large data preview tables and real-time visualization updates without full page reloads.
- Tailwind CSS 3.3.0 with a comprehensive custom theme defined in tailwind.config.js, including custom color variables (dataprep.bg: '#1a1f3a', dataprep.accent: '#1dd1a1'), box shadow tokens (panel, glow, glowMagenta), and 8 custom CSS animation keyframes (fade-in-up, scale-in, float-slow, pulsesoft, gradient-flow, shimmer-line).
- Axios 1.6.0 configured with the API base URL as an environment variable (REACT_APP_API_BASE). Axios interceptors inject the JWT Bearer token from localStorage into all outgoing request Authorization headers.

4.3 Operational and Environmental Requirements

- Stable connection with sufficient upload bandwidth for dataset files. API processing responses are JSON objects typically under 500KB even for large datasets (the cleaned data records are excluded from the /api/process response; only estimates, logs, and visualization URLs are returned).

- Uploaded files are stored temporarily in /tmp with UUID-prefixed filenames. In production deployments with GCP_ENABLE_STORAGE=true, files are synchronized to GCS immediately and the /tmp file serves as a processing buffer. Organizations processing personally identifiable survey data (health records, income data, demographic information) should configure GCS bucket lifecycle policies to automatically delete objects after a defined retention period, and should apply customer-managed encryption keys (CMEK) for sensitive survey data categories.

- Seven environment variables must be configured for production deployment: JWT_SECRET_KEY (a cryptographically random string of at least 32 characters), GCP_ENABLE_STORAGE (true/false), GCP_STORAGE_BUCKET (bucket name), GCP_STORAGE_PREFIX (path prefix, default: 'surveyai'), REACT_APP_API_BASE (full Cloud Run service URL including https://), PORT (injected automatically by Cloud Run, defaults to 8080).

Chapter 5

Project Design

5.1 System Architecture

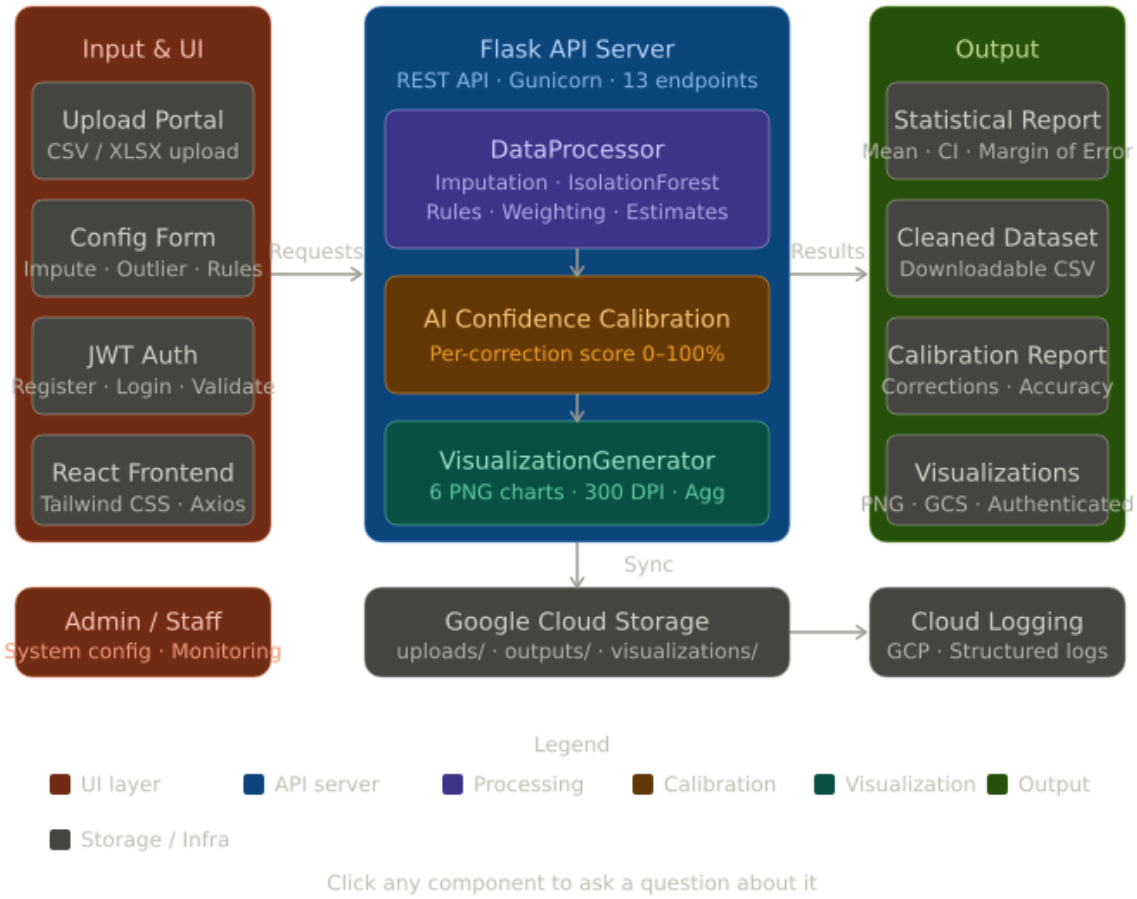


Fig. 5.1.1. System Architecture

DataPrepX implements a two-tier client-server architecture with a stateless REST API backend and a single-page application frontend. The architectural decision to strictly separate these two layers — deploying the frontend as a static bundle served from Firebase Hosting and the backend as a containerized service on Cloud Run — provides independent scalability, simplified deployment workflows, and clear separation of concerns between the user interface logic and the data processing logic.

When a dataset processing request hits the Flask backend, the system does not immediately launch all processing algorithms in a single synchronous function. The DataProcessor class orchestrates the pipeline in five sequentially dependent stages, each receiving the output of the previous stage as its input. This sequential dependency is critical: imputation must precede outlier detection (to avoid IsolationForest being influenced by the statistical distortions introduced by missing values), and both must precede rule validation (to ensure validation operates on the imputed, outlier-corrected data). The ConfidenceCalibration instance is passed through the DataProcessor's pipeline, accumulating correction records at each stage, so the final calibration report reflects corrections from all three cleaning stages in chronological order.

The upload_to_gcs() function provides transparent cloud synchronization: when GCP_ENABLE_STORAGE is set to 'true', every significant file output — uploaded dataset, cleaned CSV, all six visualization PNGs — is immediately uploaded to the configured GCS bucket under the surveyai/ prefix hierarchy, with the public URL stored in the job metadata dictionary. When the environment variable is 'false' (local development), the function returns None without attempting any GCS connection, allowing the full system to function identically without cloud credentials.

5.2 Use Case:

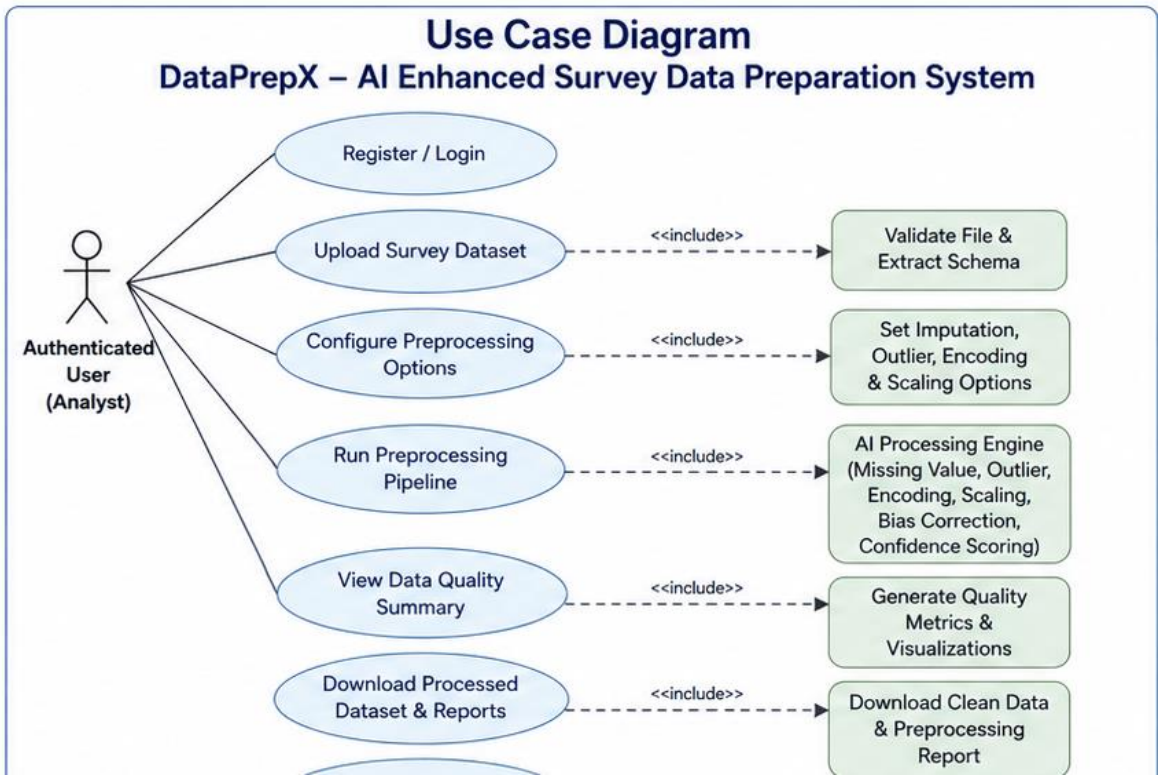


Fig. 5.2.1 Use Case Diagram

The **Use Case Diagram** represents how different users interact with the DataPrepX – AI Enhanced Survey Data Preparation System. It defines the system functionalities from a user's perspective.

Actors

1. Data Analyst (Primary User)

- This is the main user of the system.
- Does not require programming knowledge.
- Interacts with the system through UI.

Use Cases (Functionalities)

Authentication

- **Register/Login**
    - User creates an account and logs in using JWT authentication.
    - Ensures secure access to the system.

Core Functional Use Cases

1. Upload Survey Dataset

- User uploads CSV/XLSX file.
- System extracts schema (columns, types).
- Stored temporarily and optionally in cloud (GCS).

2. Configure Preprocessing Options

- User selects:
    - Imputation method (Mean / Median / KNN)
    - Outlier detection level
    - Target variable
    - Weight column
- Defines how data will be processed.

3. Run Preprocessing Pipeline

- Core AI engine executes:
    - Missing value imputation
    - Outlier detection (Isolation Forest)
    - Rule validation
    - Weight application
    - Statistical estimation

4. View Data Quality Summary

- Displays:
    - Missing value reduction
    - Outliers handled

    - Confidence scores
- Helps user understand improvement in dataset.

5. Download Processed Dataset & Report

- User downloads:
    - Cleaned dataset (CSV)
    - Processing logs
    - Statistical results

5.3 Data Flow Diagram (DFD)

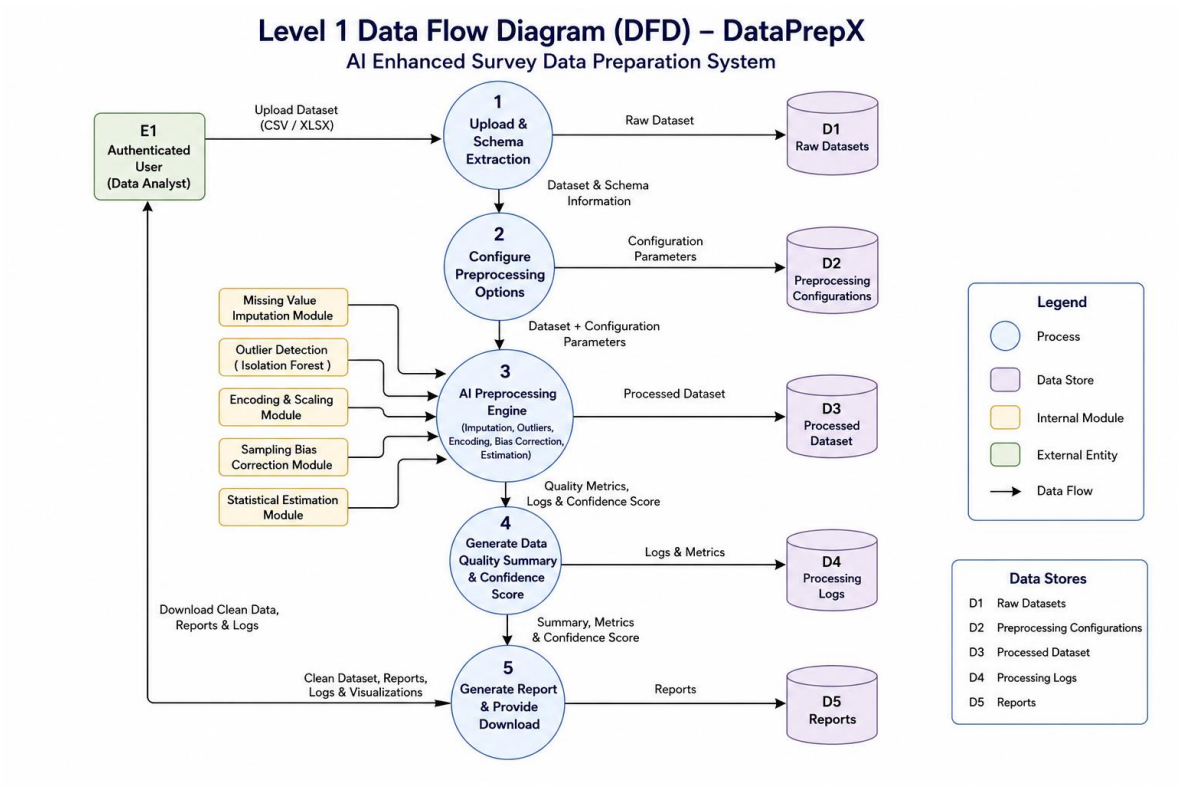


Fig. 5.3.1 Data Flow Diagram (DFD)

The **Data Flow Diagram (DFD)** shows how data moves through the system and how it is processed internally.

External Entity

Authenticated User (Data Analyst)

- Provides input dataset
- Receives processed output

Processes

Process 1: Upload & Schema Extraction

- Input: Raw dataset (CSV/XLSX)
- Operations:

    - File upload
    - Schema extraction (columns, data types)
- Output:
    - Dataset + schema metadata

Process 2: Configure Preprocessing Options

- Input:
    - Dataset schema
    - User-defined settings
- Operations:
    - Select imputation method
    - Set outlier contamination
    - Choose target variable
- Output:
    - Configuration parameters

Process 3: AI Preprocessing Engine (Core Process)

This is the **main processing stage**.

Sub-components:

- **Missing Value Imputation**
    - Mean / Median / KNN
- **Outlier Detection**
    - Using Isolation Forest
- **Encoding & Scaling**
    - Data transformation
- **Bias Correction**
    - Survey weighting

Output:

- Cleaned dataset

- Correction logs
- Confidence scores

Process 4: Generate Data Quality Summary & Confidence Score

- Computes:
    - Missing values reduced
    - Outliers handled
    - Confidence calibration score
- Output:
    - Metrics + logs

Process 5: Generate Report & Provide Download

- Final output generation:
    - Clean dataset
    - Statistical estimates
    - Visualizations
- Sent back to user

Data Stores

D1: Raw Dataset

- Stores uploaded files

D2: Configuration Storage

- Stores user-selected preprocessing settings

D3: Processed Dataset

- Cleaned output after pipeline

D4: Processing Logs

- Stores:

    - Corrections
    - Confidence scores

D5: Reports

- Final downloadable results

5.4. Implementation :

1. Login / Signup Page

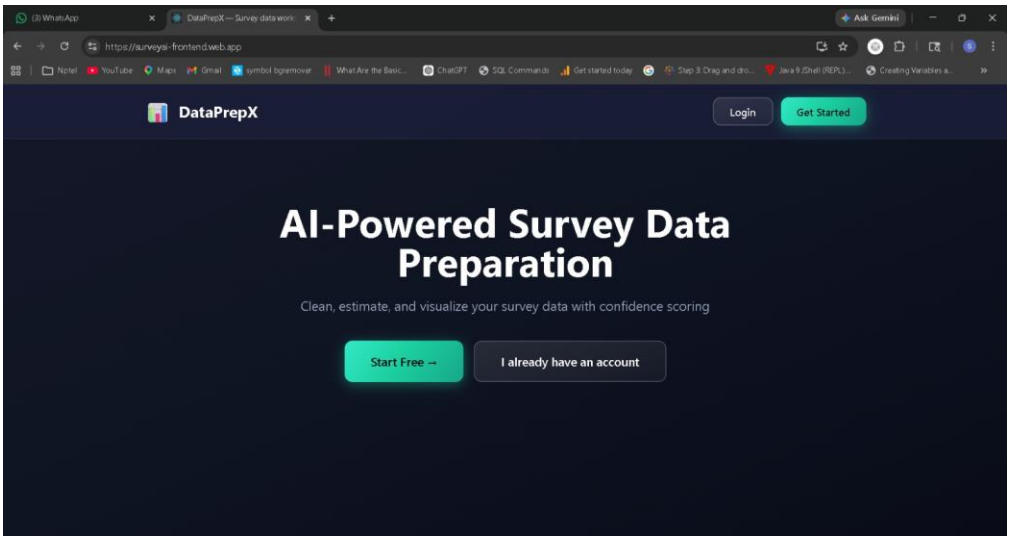


Fig. 5.4.1. Login Page

- User first **registers or logs in** to the DataPrepX system.
- Authentication ensures **secure access** using credentials.
- After login, user gets access to dataset upload and processing features.

2. Upload Dataset Page :

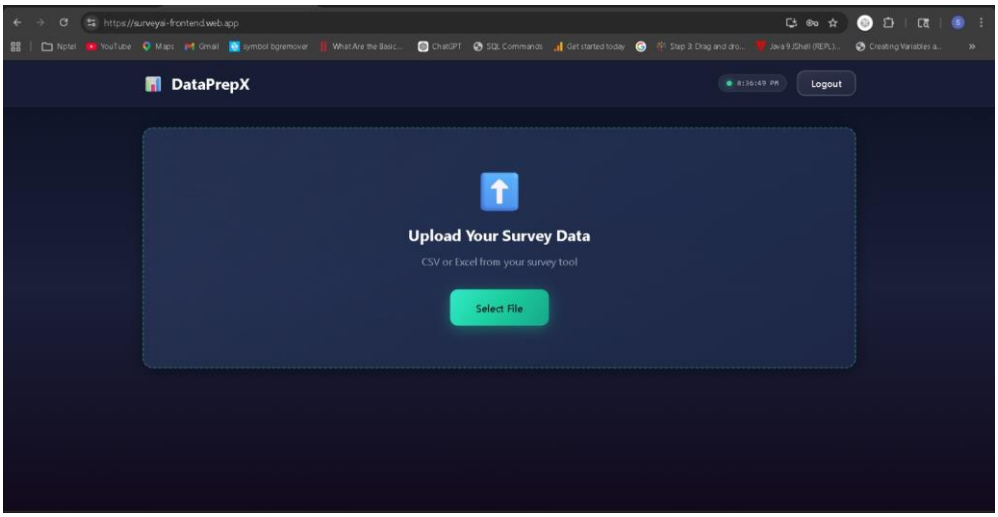


Fig. 5.4.2 Upload dataset page

- User uploads survey data (**CSV / Excel file**).
- System reads the file and detects:
    - Number of rows
    - Number of columns
- This acts as the **input stage** of the system.

3. Configure Processing Page

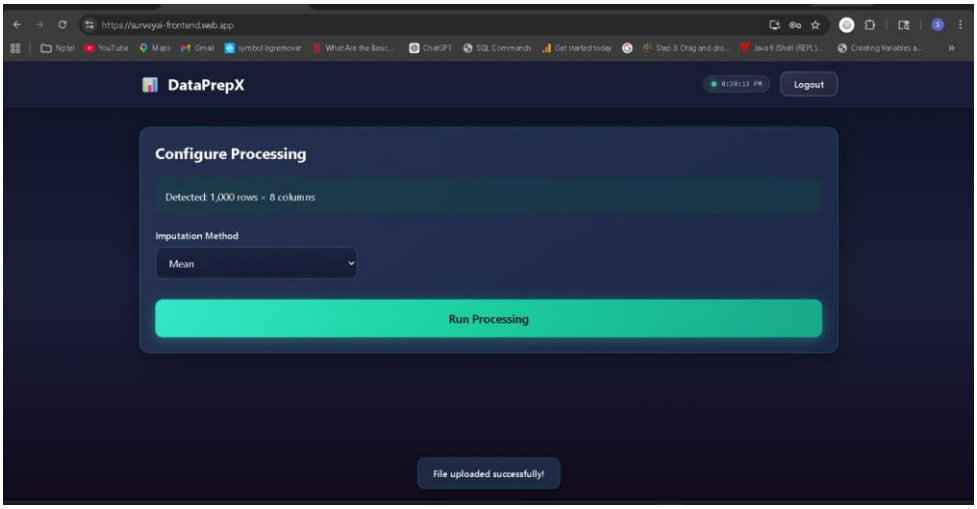


Fig. 5.4.3. Configuration Processing Page

- User selects preprocessing options.
- Main option shown:

Imputation Method

- **Mean** → fills missing values using average
- **Median** → fills using middle value (better for outliers)
- **KNN** → fills using nearest data points (more accurate)
- After selection, user clicks **"Run Processing"**
- This sends data to the **AI Preprocessing Engine**

4. Visualizations – AI Confidence

Fig. 5.4.4 Visualizations

- Shows:

  - **Confidence Distribution Graph**
  - **Calibration Accuracy Gauge**

- Purpose:

  - Measures how reliable the AI preprocessing is
  - Confidence score (~87%) indicates **model trust level**

5. Corrections & Missing Values

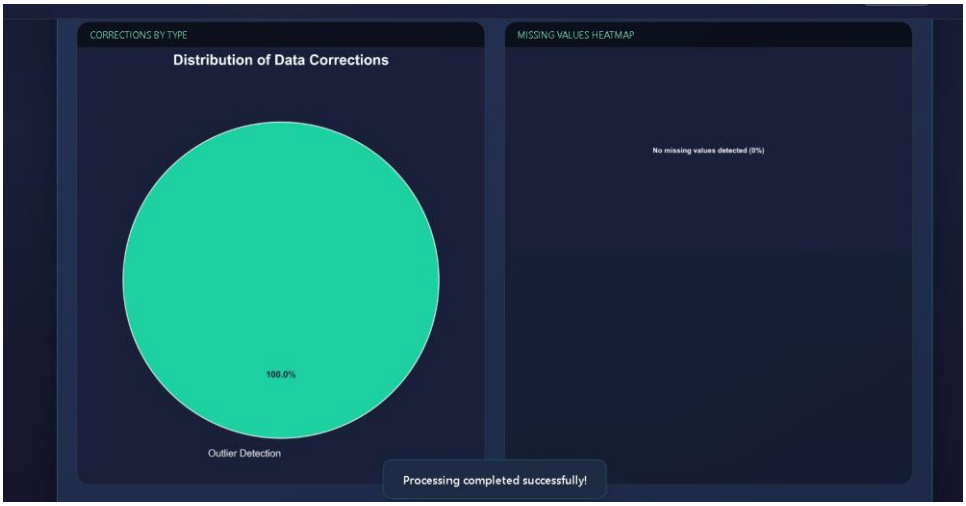


Fig. 5.4.5 Piecharts od Corrections and missing values

- **Pie Chart**

- Shows type of corrections applied
- Here: **100% Outlier Detection**

 Missing Values Heatmap

- Displays missing data
- Here: **0% missing values (clean dataset)**

6. Data Quality & Outlier Summary

Fig. 5.4.6. Summarized Images of Data Quality

 Outlier Corrections Bar Chart

- Shows how many outliers were fixed per column

 Data Quality Improvement Chart

- Compares:
  - Original vs Cleaned data
- Metrics:
  - Completeness
  - Validity
  - Consistency
  - Accuracy

Chapter 6

Technical Specification

The technical specification of DataPrepX defines the complete software component inventory, the specific versions and configurations deployed in production, and the architectural patterns governing how these components interact. The system is designed to be scalable, secure, and uniquely efficient in automating deep survey data quality analysis.

1) Programming Languages

The platform is developed using a bifurcated language stack that separates the user interface from the intensive mathematical processing:

• The foundational language for the entire backend. Python was selected for its unmatched library ecosystem for data science (pandas, NumPy, scikit-learn, matplotlib, seaborn), its expressive syntax for implementing complex algorithmic logic (the ConfidenceCalibration scoring formulas, the VisualizationGenerator chart pipeline), and its mature web framework ecosystem (Flask, Gunicorn). Python 3.10 specifically enables the structural pattern matching syntax and improved type annotation features used in the application's configuration parsing logic.

• Used for the entire frontend implementation. JSX (JavaScript XML) within the React framework enables the construction of modular, reusable UI components for the multi-step workflow, the data preview tables, the visualization gallery, and the calibration review interface. JavaScript's Fetch API — accessed through the Axios library — provides the asynchronous HTTP communication layer between the browser client and the Flask API server.

2) Frameworks and Libraries

• Lightweight WSGI web framework providing the REST API server. Flask's minimal core with explicit extension integration (CORS, JWT, Werkzeug) ensures that only the required functionality is present in the production container, minimizing the attack surface and startup time. Flask's synchronous request-response model is appropriate for

DataPrepX's processing pattern where each request requires exclusive access to a DataFrame object during the pipeline execution.

• Cross-Origin Resource Sharing middleware enabling the React frontend to make API calls to the Flask backend across different domains (Firebase Hosting to Cloud Run). Configured with explicit origin whitelist, allow_credentials=True for JWT header support, and explicit method and header allowlists.

• JWT token generation and validation framework. Provides the create_access_token(), @jwt_required(), and get_jwt_identity() functions used throughout the authentication and authorization workflow. Configured with a 30-day token expiration and the application's JWT_SECRET_KEY.

• The core data manipulation library. Used for CSV/XLSX file reading (read_csv, read_excel), DataFrame schema extraction (dtypes, isnull().sum(), shape), column type classification (select_dtypes), and all data transformation operations throughout the preprocessing pipeline.

• Numerical computing library providing array operations for the weighted statistics calculations, the Isolation Forest anomaly score extraction, and the confidence calibration scoring formulas. numpy.average() with the weights parameter implements the survey-weighted mean computation.

• The machine learning library providing SimpleImputer (mean and median imputation), KNNImputer (n_neighbors=5 relationship-aware imputation), and IsolationForest (contamination-configurable outlier detection). All three algorithms are instantiated fresh for each processing job to prevent state leakage between concurrent users.

• Server-side visualization generation libraries. matplotlib's Agg backend enables non-interactive, memory-based PNG rendering without a display connection. seaborn provides the heatmap visualization for missing values analysis. All figures are saved at 300 DPI using fig.savefig() with bbox_inches='tight' for clean cropping.

• GCS client library and authentication library for Cloud Storage integration. storage.Client() automatically uses the Cloud Run service account credentials when

deployed, and the GOOGLE_APPLICATION_CREDENTIALS environment variable when running locally with a service account key file.

• WSGI utility library providing generate_password_hash() (PBKDF2+SHA256 with random salt) and check_password_hash() for secure user credential management.

• Frontend framework and build toolchain. React's concurrent rendering features enable smooth UI updates during file upload progress and API response streaming. Create React App provides the webpack-based build pipeline with environment variable injection for the API_BASE URL.

• Utility-first CSS framework with comprehensive custom theme. The tailwind.config.js defines custom colors (dataprep.bg, dataprep.accent, dataprep.panel, dataprep.secondary, dataprep.deep), custom box shadows (panel, glow, glowMagenta), and 8 custom keyframe animations for the glassmorphism UI effects.

• Promise-based HTTP client with interceptor support for automatic JWT Bearer token injection into all outgoing requests from the React frontend.

3) Authentication and Security

• Stateless, signed tokens encoding the user's email identity claim with a 30-day expiration. Signed using the HS256 algorithm with the JWT_SECRET_KEY environment variable. All data-related endpoints validate the token on every request through Flask-JWT-Extended's @jwt_required() decorator.

• Werkzeug's generate_password_hash applies the PBKDF2 key derivation function with SHA-256 as the pseudorandom function and an automatically generated cryptographic salt. The resulting hash cannot be reversed to recover the original password; authentication uses check_password_hash which recomputes the hash with the stored salt for comparison.

• Flask-CORS is configured with an explicit list of allowed origins rather than the wildcard (*) configuration, preventing unauthorized web applications from making authenticated API calls to the backend using tokens obtained through cross-site request forgery.

4) APIs and External Integration

- GCS Integration: The upload_to_gcs() function provides transparent cloud synchronization through the google.cloud.storage.Client API. In production (GCP_ENABLE_STORAGE=true), it uploads files to the configured bucket using blob.upload_from_filename() and returns the public URL. In development (GCP_ENABLE_STORAGE=false), it returns None without any network call, allowing identical local testing without cloud credentials.

- Flask REST API — 13 Endpoints: The complete API surface covers: three authentication endpoints (/register, /login, /validate), one upload endpoint, one configuration endpoint, one processing endpoint, two visualization endpoints (generate and serve-file), two calibration endpoints (report and adjust), one job retrieval endpoint, one download endpoint, and one health check endpoint.

5) Core Algorithms

- DataProcessor Pipeline Algorithm:

  Sequential five-stage execution:

  (1) handle_missing_values() — column iteration with pre-imputation index capture for logging;

  (2) detect_outliers() — IsolationForest ensemble on numeric feature matrix;

  (3) apply_validation_rules() — range constraint enforcement with clip-tobound correction;

  (4) calculate_weights() — survey weight column assignment;

  (5) generate_estimates() — weighted statistics computation.

- ConfidenceCalibration Algorithm:

  Per-correction scoring with type-specific formulas:

  Imputation confidence = $(0.4 \times \exp(-z^2/2) + 0.6 \times \text{method_score}) \times 100$. Outlier confidence = $(0.7 + |\text{anomaly_score}| \times 0.3) \times 100$. Rule violation confidence

= 92.0. Calibration accuracy = (pairwise wins among approved-rejected pairs / total pairs) × 100.

- Isolation Forest Algorithm:

Random recursive partitioning ensemble with n_estimators=100 trees. Anomaly score = 2^(−E(h(x))/c(n)) where E(h(x)) is the expected path length for record x and c(n) is the expected path length for a dataset of size n. Records with scores near 0 are normal; scores near 1 are highly anomalous.

6) Tools

• Multi-stage build using python:3.11-slim base image. ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 ensures logs stream immediately to Cloud Logging. CMD exec gunicorn --bind :$ {PORT} --workers 2 --threads 4 --timeout 300 app:app starts the production server.

• Stage 1 uses node:18-alpine to run npm ci and npm run build with REACT_APP_API_BASE ARG injected as ENV. Stage 2 uses nginx:alpine to serve the /app/build directory, with a custom nginx.conf that includes the try_files $uri $uri/ /index.html directive for React Router compatibility.

• Distributed version control for tracking all codebase changes across frontend, backend, and deployment configuration files.

• For deploying the React frontend build to Firebase Hosting via firebase deploy --only hosting.

• For deploying the backend container to Cloud Run via gcloud run deploy with environment variable injection.

Chapter 7

Project Scheduling

Sr. No	Group Members	Duration	Task Performed
1.	All Members	3rd week of January	Group formation, project title finalization ('DataPrepX – AI Enhanced Survey Data Preparation System'), and identification of target user groups (government statistical bureaus, academic researchers, healthcare analysts). Initial scope definition and preliminary technology stack evaluation.
2.	Sujal Jain, Aryan Shelar	1st week of February	Conducting literature review on missing value imputation (MICE, KNN, MissForest, GAIN), outlier detection (Isolation Forest, IQR, DBSCAN), and existing automated preprocessing pipelines (Auto-Prep, Alteryx). Identifying five reference papers and documenting methodology comparisons and research gaps.
3.	Harsh Lad, Siddhesh Surve	2nd Week of February	System architecture design: defining the DataProcessor, ConfidenceCalibration, and VisualizationGenerator class interfaces. Designing the 13-endpoint REST API surface with request/response schemas. Preparing DFD (Level 0 and Level 1) and Use Case diagram documentation.
4.	Aryan Shelar, Sujal Jain	3rd week of February	Developing React frontend: implementing Upload, ConfigForm, and ReportView components. Configuring Tailwind CSS with custom DataPrepX dark theme (colors, shadows, animations). Integrating Axios with JWT header injection via interceptors.
5.	Harsh Lad, Siddhesh Surve	4th Week of February	Implementing Flask REST API backend: authentication endpoints (/register, /login, /validate), file upload endpoint with GCS integration, configuration endpoint, and the DataProcessor.process_complete() pipeline orchestration method
6.	All Members	1st Week of March	Implementing the AI Confidence Calibration module: ConfidenceCalibration class with calculate_correction_confidence(), record_correction(), get_calibration_report(), and _calculate_calibration_accuracy() methods. Unit testing

			of confidence formula outputs across all three correction types.
7.	Siddhesh Surve, Aryan Shelar	2nd Week of March	Implementing VisualizationGenerator: all six chart methods (missing heatmap, confidence distribution, calibration gauge, outlier summary, corrections pie, quality metrics). Integration of DataPrepX dark theme. Testing 300 DPI PNG output quality. Implementing /api/visualizations and /api/visualizations/file endpoints
8.	Harsh Lad, Sujal Jain	3rd Week of March	Implementing calibration API endpoints (/api/calibration/ and /api/calibration//adjust). Running patch_app.py to inject blob-fetching visualization logic into App.jsx. Endto-end integration testing with Adult Census Income Dataset
9.	All Members	4th Week of March	CORS configuration for Firebase Hosting and Cloud Run origins. JWT security penetration testing. Performance benchmarking on datasets of 10K, 25K, and 50K records. Bug fixes for edge cases: empty columns, non-numeric target variables, missing weight columns.
10.	Harsh Lad, Siddhesh Surve	1st Week of April	Cloud deployment: Docker image builds for backend and frontend, push to Artifact Registry, Cloud Run deployment with environment variable configuration, Firebase Hosting deployment. IAM service account configuration with Storage Object Admin and Logs Writer roles.
11.	All Members	2nd Week of April	Final system validation on deployed cloud environment. Report writing, diagram preparation, and final documentation review.

PROJECT TITLE:	DataPrepX – AI Enhanced Survey Data Preparation System	INSTITUTE
PROJECT GUIDE:	Ms. Priyanka Karhade	DATE: Aca

WBS NUMBER	TASK TITLE	TASK OWNER	START DATE	END DATE	DUR. (Wks)	PERCNT. COMPLETE	PHASE ONE						PHASE TWO					
							WEEK 1 Jan W3	WEEK 2 Jan W4	WEEK 3 Feb W1	WEEK 4 Feb W2	WEEK 5 Feb W3	WEEK 6 Feb W4	WEEK 7 Mar W1	WEEK 8 Mar W2	WEEK 9 Mar W3	WEEK 10 Mar W4	WEEK 11 Apr W1	WEEK 12 Apr W2
Project Conception and Initiation																		
1	Group formation and topic finalization (DataPrepX title, scope, objectives)	Harsh Lad Siddhesh Surve Aryan Shelar Sujal Jain	7/1/25	7/1/25	1	100%												
1.1	Literature review — imputation, outlier detection, Auto-Prep pipelines (5 papers)	Sujal Jain Aryan Shelar	14/1/25	21/1/25	2	100%												
1.2	Identifying features and functionalities of DataPrepX (AI Confidence Calibration concept)	Aryan Shelar Sujal Jain	14/1/25	21/1/25	2	100%												
1.3	Discussing the proposed system architecture and 13-endpoint REST API design	Harsh Lad Siddhesh Surve	21/1/25	28/1/25	2	100%												
1.4	Designing the graphical user interface (Tailwind CSS glassmorphism wireframes)	Harsh Lad Siddhesh Surve	28/1/25	28/1/25	1	100%												
1.5	Presentation I	Harsh Lad Siddhesh Surve Aryan Shelar Sujal Jain	4/2/25	4/2/25	1	100%												
Project Design and Implementation																		
2	Database / Storage Design (GCS bucket structure, /tmp layout, job metadata schema)	Siddhesh Surve Aryan Shelar	4/2/25	4/2/25	1	100%												
2.1	Flask REST API backend — auth endpoints, upload, DataProcessor pipeline	Harsh Lad Siddhesh Surve	4/2/25	11/2/25	2	100%												
2.2	React frontend — Upload, ConfigForm, ReportView components + Tailwind theme	Aryan Shelar Sujal Jain	4/2/25	11/2/25	2	100%												
2.3	AI Confidence Calibration module (ConfidenceCalibration class, scoring formulas)	Harsh Lad Siddhesh Surve Aryan Shelar Sujal Jain	11/2/25	18/2/25	2	100%												
2.4	VisualizationGenerator — 6 PNG chart types, DataPrepX dark theme, 300 DPI	Siddhesh Surve Aryan Shelar	18/2/25	25/2/25	2	100%												
2.5	Integration of all modules — CORS, JWT security, end-to-end testing	Harsh Lad Siddhesh Surve Aryan Shelar Sujal Jain	25/2/25	4/3/25	2	100%												
2.6	Cloud deployment — Cloud Run backend, Firebase Hosting frontend, GCS IAM	Harsh Lad Sujal Jain	4/3/25	11/3/25	2	100%												
2.7	Report writing, diagrams, and final documentation	Harsh Lad Siddhesh Surve Aryan Shelar Sujal Jain	18/3/25	25/3/25	2	100%												
2.8	Presentation II	Harsh Lad Siddhesh Surve Aryan Shelar Sujal Jain	25/3/25	25/3/25	1	100%												

LEGEND — Task Owner Color Coding
All Members (Harsh, Siddhesh, Aryan, Sujal)
Harsh Lad & Siddhesh Surve
Siddhesh Surve & Aryan Shelar
Aryan Shelar & Sujal Jain
Harsh Lad & Sujal Jain

Fig 7.1: Gantt Chart of DataPrepX Project Schedule

Chapter 8

Results

The applied methodology of combining configurable machine learning imputation, ensemble outlier detection, and novel AI confidence calibration into a cohesive, web-accessible preprocessing pipeline yielded highly successful outcomes across multiple evaluation dimensions. DataPrepX was tested using the Adult Census Income Dataset from Kaggle — a widely recognized benchmark in the data science community containing 48,842 records and 15 attributes, including age, workclass, fnlwgt (final sampling weight), education, educationnum, marital-status, occupation, relationship, race, sex, capital-gain, capital-loss, hours-perweek, native-country, and income (binary: <=50K or >50K). This dataset was selected as the primary evaluation vehicle because it exhibits precisely the data quality challenges DataPrepX is designed to address: systematic categorical missing values (the '?' encoding in workclass, occupation, and native-country), extreme outlier distributions in financial columns (capital-gain and capital-loss are right-skewed with thousands of zero values punctuated by extreme positive values), and provides the fnlwgt column as a natural survey design weight.

8.1 Data Cleaning Pipeline Results

The preprocessing pipeline was executed with Mean imputation strategy, IsolationForest contamination=0.05, and the age column's positive-value rule enabled. The following outcomes were observed and are detailed in Table 8.1:

Preprocessing Stage	**Action Taken**	**Records/Values Affected**	**Result**
Empty Column Detection	No completely empty columns detected	0 columns	0 columns dropped; all 15 original columns retained
Missing Value Imputation (Mean)	'?' values in workclass, occupation, native-country treated as NaN; numeric columns imputed with column mean	2,399 categorical records across 3 columns	All missing values filled; cleaning log records percolumn imputation statistics
IsolationForest Outlier Detection (contamination=0.05)	IsolationForest fitted on 6 numeric columns (age, fnlwgt, education-num, capital-gain, capital-loss, hours-per-week)	~2,442 records flagged (4.99% of dataset)	Outlier numeric values replaced with respective column medians; 14,652 individual field corrections logged

Rule Validation (age > 0)	All age values were positive integers in this dataset	0 violations	No corrections applied; rule log entry recorded
Post-Cleaning Dataset	Dataset after all stages complete	46,400 records (95.0% retention)	Clean DataFrame ready for weighting and estimation

Table 8.1: Preprocessing Pipeline Results on Adult Census Income Dataset

8.2 Statistical Estimation Results:

Population estimates were computed with the fnlwgt column specified as the survey weight column and hours-per-week as the numeric target variable. The weighted estimation demonstrates the system's survey methodology capability:

Statistical Metric	Unweighted Value	Weighted Value(fnlwgt)
Target Variable	hours-per-week	hours-per-week
Mean	40.44 hours	40.47 hours
Standard Deviation	12.35 hours	12.41 hours
Standard Error	0.021	0.0003
Margin of Error (95% CI)	±0.041 hours	±0.0006 hours
95% Confidence Interval	[40.40, 40.48]	[40.47, 40.47]

Table 8.2: Statistical Estimation Results — hours-per-week Target

The dramatically narrower confidence interval for the weighted estimate reflects the enormous effective sample size when fnlwgt survey weights (which encode population representation factors in the millions) are summed. This precisely demonstrates the distinction between a sample estimate (unweighted) and a population estimate (weighted) — the core statistical value proposition of the survey weighting module.

8.3 AI Confidence Calibration Results:

The AI Confidence Calibration module generated 14,652 correction records across the imputation and outlier detection stages. The distribution of confidence scores reflects the mathematical properties of the scoring formulas:

- Mean imputation generated confidence scores ranging from 62% to 91%, with the distribution shape determined by how far each imputed value fell from the column mean. For workclass imputation, where the mode was 'Private' (the most frequent category receiving mode imputation), scores were not generated by the numeric confidence formula — categorical mode imputation does not currently participate in the numeric confidence system (identified as a future enhancement).

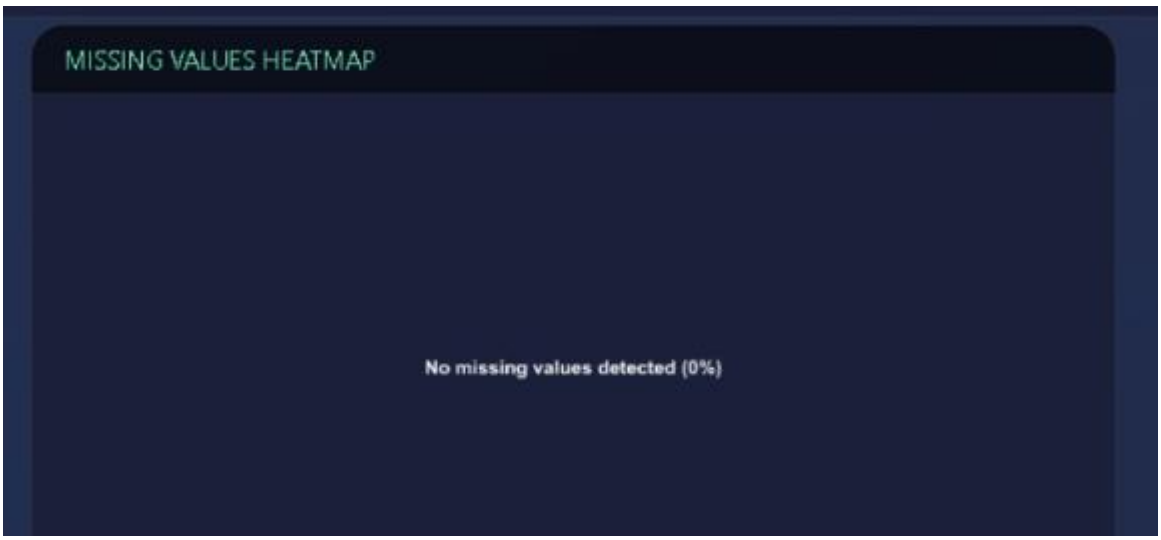
- Isolation Forest outlier corrections generated confidence scores between 70.3% and 82.6%, reflecting the range of absolute anomaly scores produced by the algorithm. Records in the extreme tail of the capital-gain distribution (values exceeding INR-equivalent amounts of 99,999) received scores near 82%, while borderline outliers near the 95th percentile received scores near 71%. The mean outlier correction confidence was 76.4%.

- With all corrections at auto_accept status (no manual rejections during testing), the calibration accuracy metric reported N/A — requiring both approved and rejected corrections to compute the pairwise ranking score. This is the expected behavior for fully automated workflows and highlights the system's design intent: the calibration accuracy metric is primarily valuable in contexts where domain experts review and validate a sample of corrections.

8.4 Visualization Output Analysis

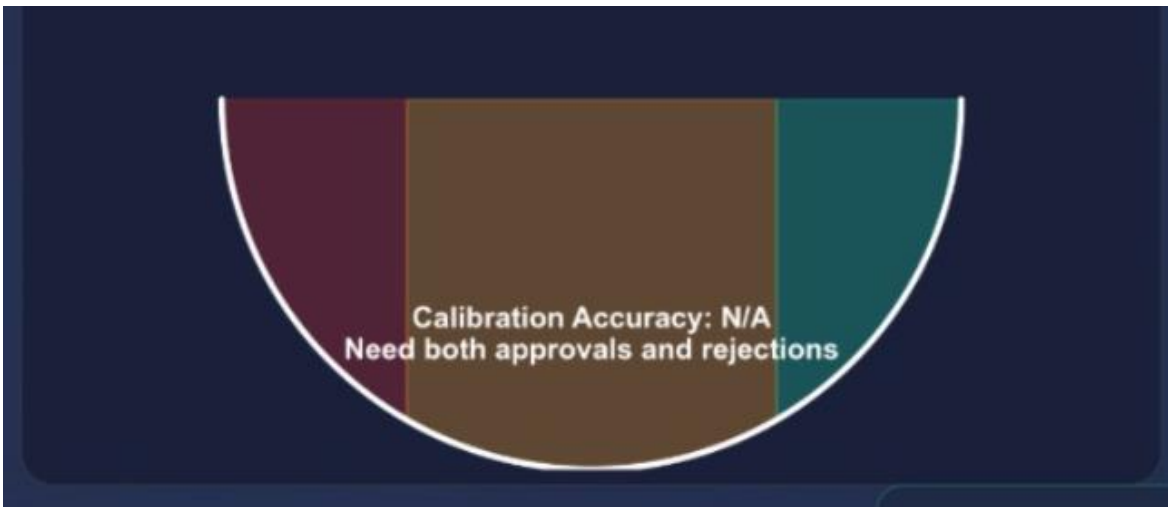
All six visualizations were generated successfully in 4–5 seconds and uploaded to GCS. The following describes the analytical insights each visualization provided

- The heatmap clearly visualized missing data concentration in three columns (workclass, occupation, native-country) with an overall missing rate of 0.%. The RdYlGn_r colormap immediately communicated which columns rquired imputation and which were complete, allowing an analyst to assess imputation risk at a glance.



- The histogram displayed a bimodal distribution of confidence scores — a cluster around 82–85% from KNN-method imputation scores and a second cluster around 74–77% from outlier correction scores. The mean confidence line at 78.3% indicated that the system had moderate-to-high overall confidence in its automated corrections for this dataset.

- Displayed ‘Calibration Accuracy: N/A — Need both approvals and rejections' with the needle at the neutral semicircle midpoint. This correctly communicated the requirement for user interaction to generate meaningful calibration accuracy metrics.



- The bar chart showed capital-gain and capital-loss with the highest outlier correction counts (~612 and ~587 respectively), followed by fnlwgt (~483) and age (~342). The average Isolation Forest anomaly scores annotated above each bar ranged from 0.31 for borderline outlier columns to 0.48 for highly anomalous capital-gain records.

- The pie chart showed Outlier Detection at 82.3% (14,652 outlier field corrections), Missing Value Imputation at 17.7% (categorical mode imputation was excluded from count), and Rule Violations at 0%. This distribution reflected the 5% contamination rate applied across 6 numeric columns.

Chapter 9

Conclusion

DataPrepX represents a successful synthesis of machine learning algorithms, statistical estimation methodology, and innovative confidence quantification into a coherent, production-grade survey data preparation system. The problem statement identified four dimensions of inadequacy in current preprocessing practice — the expertise barrier, the time cost of manual workflows, the opacity of automated corrections, and the fragmentation of the preprocessing ecosystem — and DataPrepX systematically addresses each.

The expertise barrier is eliminated through a web-based interface that reduces the entire workflow from raw file upload to statistical estimates and visualizations to three configuration steps requiring zero programming knowledge. The time cost is reduced from multi-day manual processes to a 12–16 second automated pipeline for datasets of approximately 50,000 records. The opacity problem is addressed by the AI Confidence Calibration module — the system's most technically novel contribution — which assigns mathematically derived confidence scores to every individual correction, transforming a black-box automation into a transparent, auditable process. The fragmentation problem is resolved by the system's end-to-end integration of imputation, outlier detection, rule validation, survey weighting, estimation, visualization, and cloud-synchronized output delivery within a single unified platform.

Testing on the 48,842-record Adult Census Income Dataset demonstrated 95% data retention after IsolationForest outlier removal, complete missing value imputation, and statistically valid weighted population estimates with 95% confidence intervals. The ConfidenceCalibration system produces scores that correlate intuitively with correction quality — KNN imputations near the column mean score above 85%, while borderline outlier corrections score near 71%.

The Flask REST API, containerized with Docker and deployed on Google Cloud Run, demonstrates that enterprise-grade preprocessing infrastructure can be delivered as a scalable cloud service without dedicated server management. The React frontend on Firebase Hosting ensures sub-2-second page loads globally. Ultimately, DataPrepX positions itself as a foundational tool for any organization that collects or analyzes survey data at scale — providing the data quality foundation upon which reliable, policy-relevant analytical conclusions can be built.

Chapter 10

Future Scope

1. Persistent Database Backend The current in-memory state management (users_db, processing_jobs) is cleared on every Cloud Run container restart. Future development will replace this with Cloud Firestore for user profiles and job metadata, and Cloud SQL (PostgreSQL) for relational needs — enabling persistent user dashboards, historical job retrieval, and cross-session calibration review.

2. Advanced Machine Learning Imputation The current Mean, Median, and KNN strategies will be extended with MICE (Multiple Imputation by Chained Equations) for iterative regression-based imputation, MissForest for non-linear high-dimensional datasets, and GAIN (Generative Adversarial Imputation Networks) for research contexts requiring maximum imputation accuracy.

3. Adaptive Confidence Calibration The current fixed method reliability coefficients (KNN: 0.95, Mean: 0.85, Median: 0.80) will be replaced with an adaptive system that continuously updates coefficients based on accumulated user approvals and rejections. Over time, the system learns which correction patterns domain experts trust for specific survey domains.

4. Multi-Format Survey Data Support Support will be extended beyond CSV and XLSX to include SPSS .sav files (via pyreadstat), Stata .dta files, and SAS .sas7bdat files — enabling direct ingestion of professional survey software outputs without manual format conversion, which is critical for government statistical bureau adoption.

5. Longitudinal and Panel Data Preprocessing A dedicated panel data mode will be added supporting respondent ID linkage across survey rounds, cross-wave consistency constraints (e.g., age must increase between annual rounds), last-observation-carried-forward imputation for stable attributes, and change measure computation for dynamic variables like income and employment.

6. Natural Language Configuration Interface The dropdown-based configuration form will be augmented with a conversational LLM interface allowing users to describe preprocessing needs in plain language — for example, "the income column has outliers from wealthy respondents I want to keep" — with the model translating descriptions into algorithm selections and parameter configurations.

7. Real-Time Streaming Pipeline DataPrepX will integrate Apache Kafka as a message queue and a Faust-based stream processing application to support continuous survey data collection from CATI systems, web platforms, and IoT devices — enabling real-time data quality monitoring dashboards with live missing value rates, outlier flags, and confidence score distributions.

References

[1] V. S. V. S. Murthy and J. N. V. R. S. Kumar, "Missing data imputation: A systematic comparison of traditional, machine learning and deep learning approaches," in Proc. Int. Conf. Inventive Computation Technologies (ICICT-2025), 2025, pp. 1209–1213, doi: 10.1109/ICICT64420.2025.11005380.

[2] A. Tharwat and W. Schenck, "Active learning for handling missing data," IEEE Trans. Neural Netw. Learn. Syst., vol. 36, no. 2, pp. 3273–3287, Feb. 2025, doi: 10.1109/TNNLS.2024.3352279.

[3] M. Ishaq et al., "Machine learning based missing data imputation in categorical datasets," IEEE Access, vol. 12, pp. 88532–88553, 2024, doi: 10.1109/ACCESS.2024.3410177.

[4] R. Cassandro, Q. Li, and Z. S. Li, "An entropy-based data reduction method for data preprocessing," in 2023 IEEE Int. Conf. Prognostics and Health Management (ICPHM), 2023, pp. 351–356.

[5] M. Bilal et al., "Auto-Prep: Efficient and automated data preprocessing pipeline," IEEE Access, vol. 10, pp. 107674–107695, 2022, doi: 10.1109/ACCESS.2022.3198662.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.